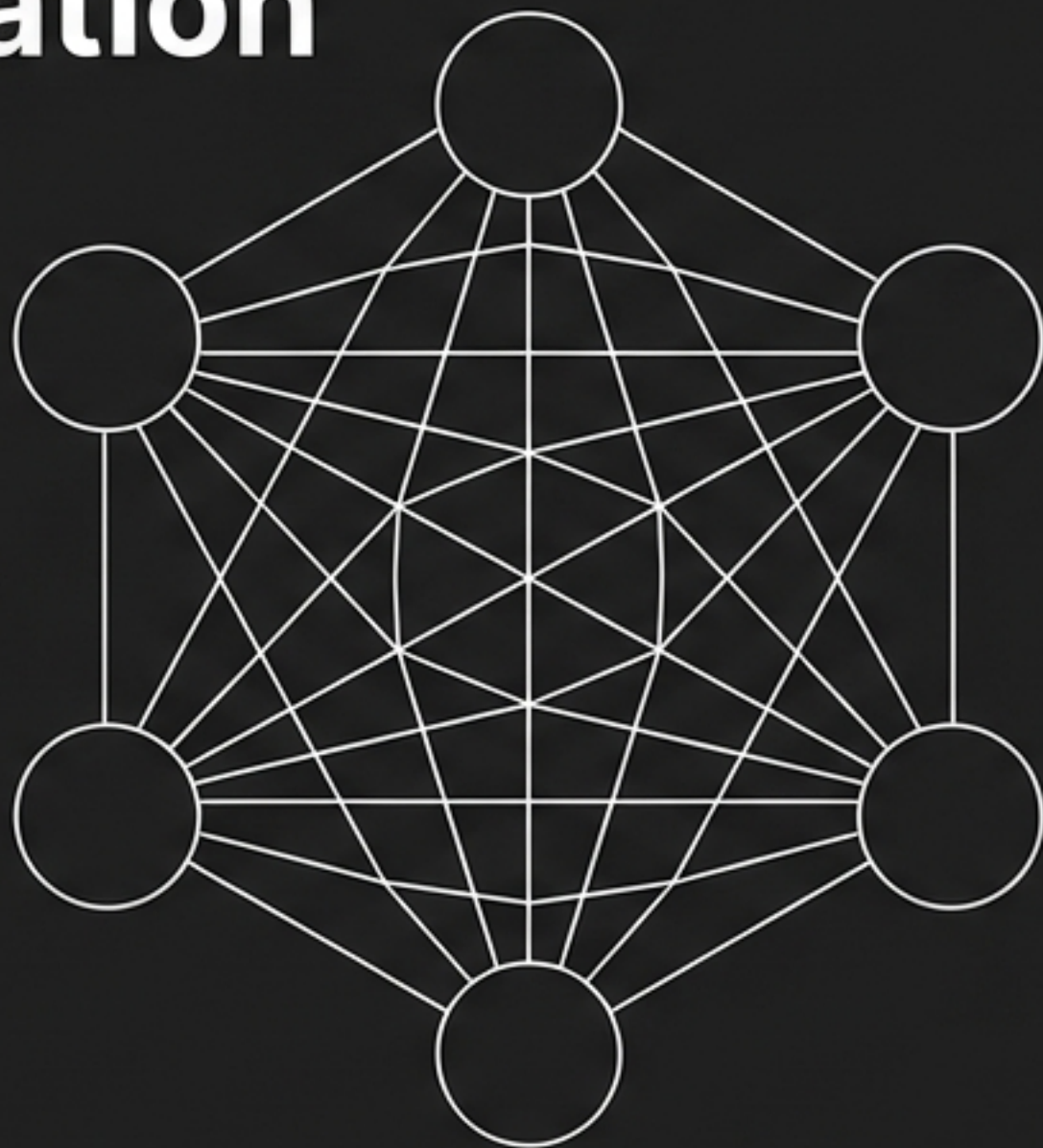


Issues-FS: Role-Based Agent Coordination

A submodule architecture for specialized, self-managing AI teams.



Version: v1.0

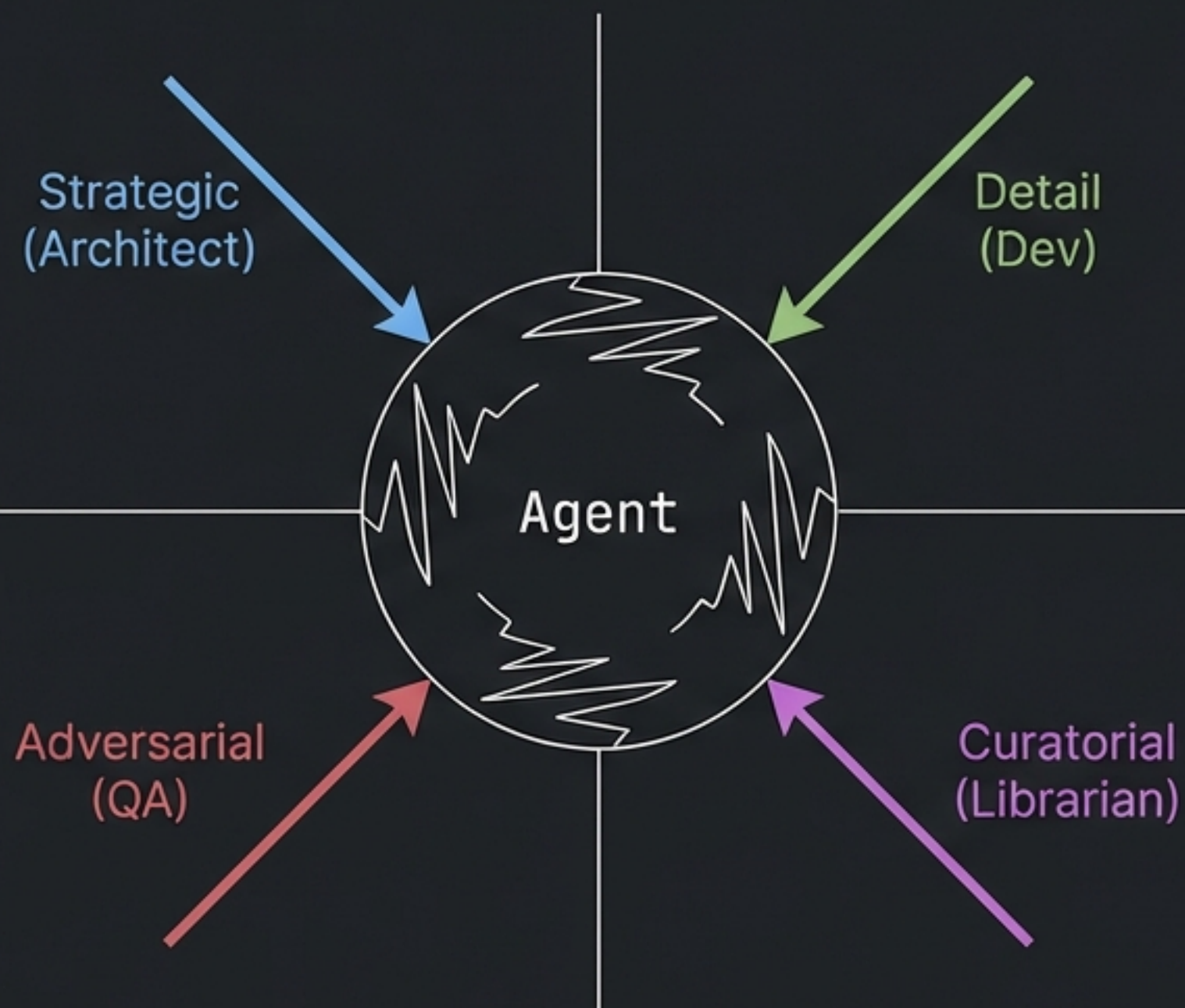
Date: 05 Feb 2026

Status: Draft

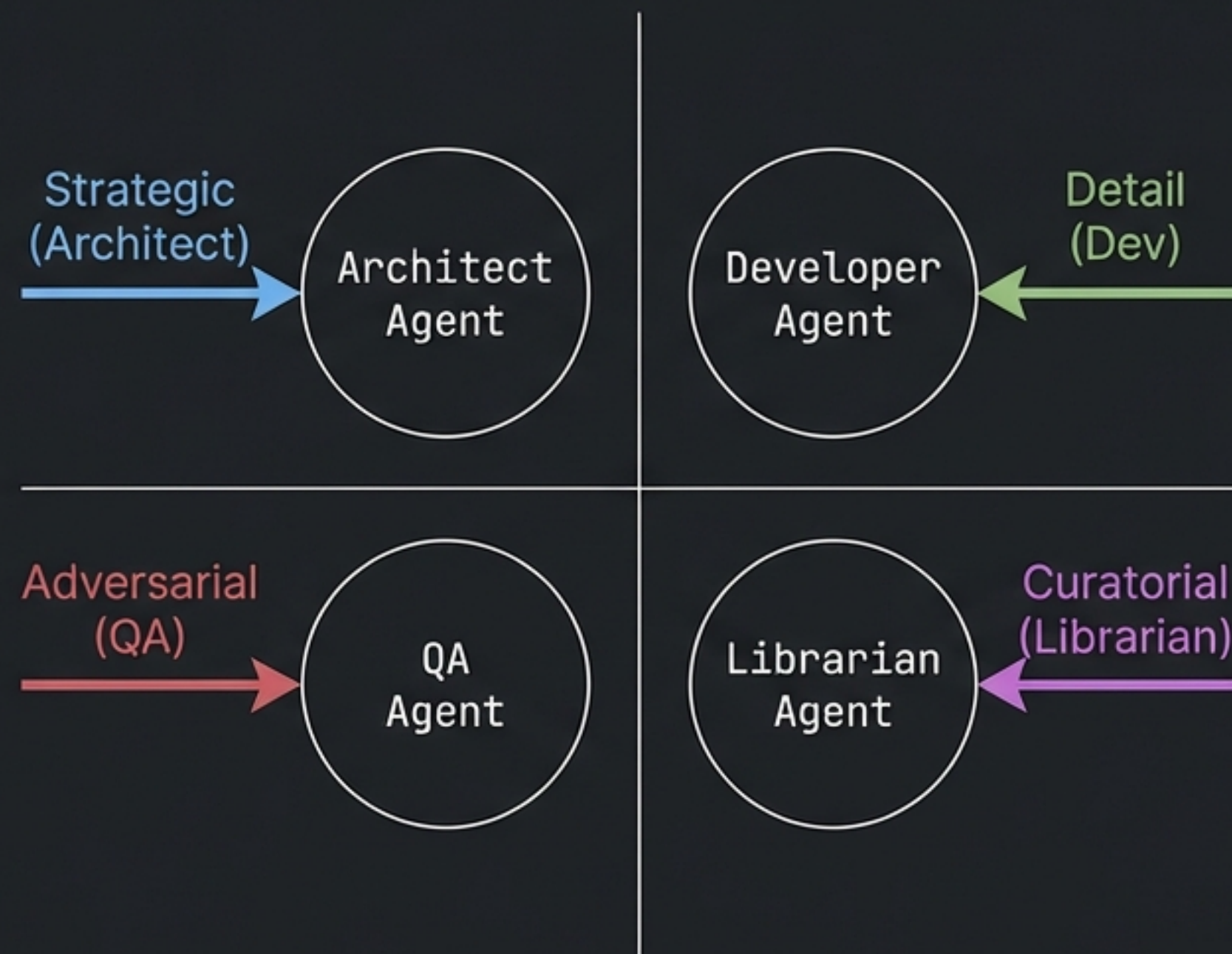
Dependency: Issues-FS__Architecture-Overview v1.0

The Problem: Context Pollution

Monolithic Context



Role Specialization



Core Argument: When a single agent attempts to wear multiple hats—writing code, reviewing it, and managing releases—quality degrades. Team members are instances of the same model, differentiated only by context.

The Solution: The 'Role Repo' Pattern

```
Issues-FS__Dev/
├── modules/           # Source Code
│   └── ...
├── roles/             # Agent Definitions
│   ├── Issues-FS__Dev__Role__Conductor/
│   │   └── ...
│   ├── Issues-FS__Dev__Role__Dev/
│   │   └── ...
│   ├── Issues-FS__Dev__Role__QA/
│   │   └── ...
│   └── Issues-FS__Dev__Role__Architect/
│       └── ...
└── ...
```

The Insight: The submodule pattern works for agent roles just as well as it works for code.

A "**Role Repo**" is a repository where the primary artifact is agent configuration, not just source code.

The Ensemble: Six Foundational Roles



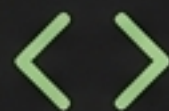
Conductor

Orchestration,
Blockers, Sprint
Planning.



Architect

Strategy, API
Design, ADRs.



Dev

Implementation,
Source Code, Unit
Tests.



QA

Quality Gates,
Integration Tests,
Defects.



DevOps

CI/CD,
Infrastructure,
Releases.



Librarian

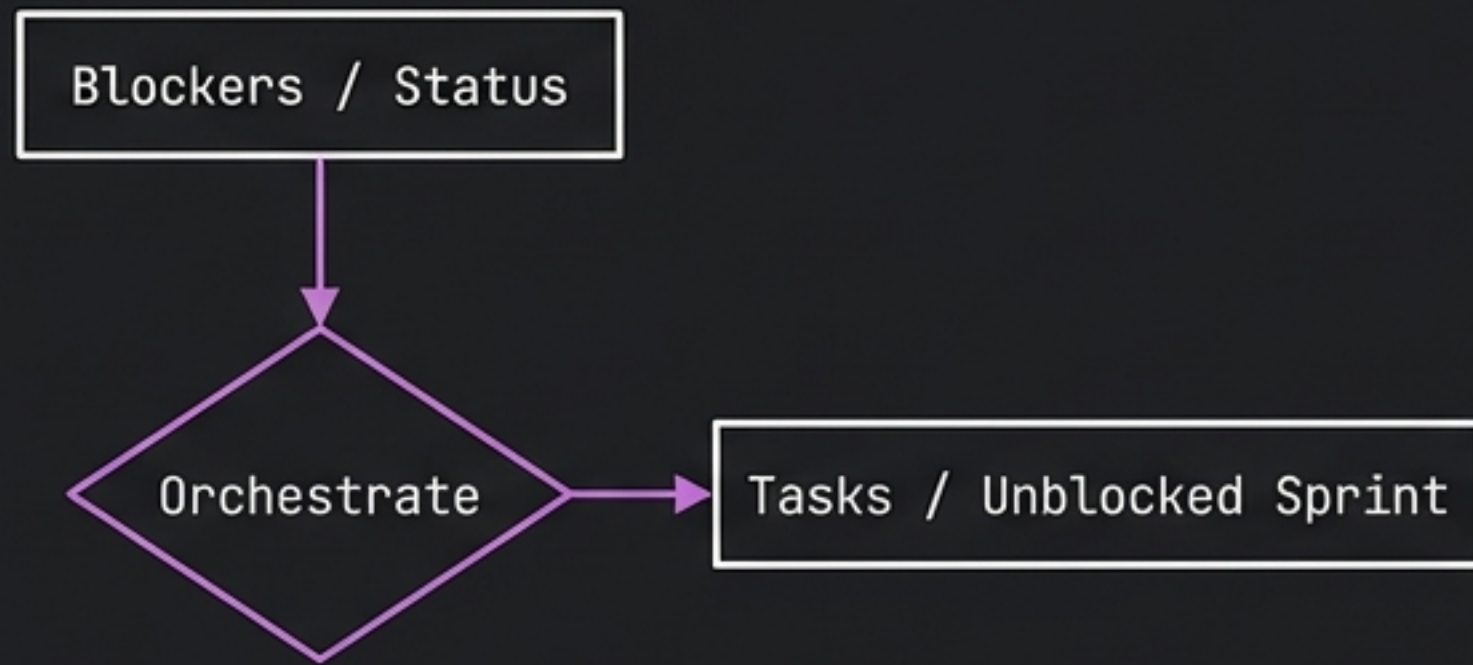
Documentation,
Curation, "Truth".

These roles cover the core development workflow. Each has a clear, non-overlapping scope.

Strategic Roles: Conductor & Architect

The Conductor

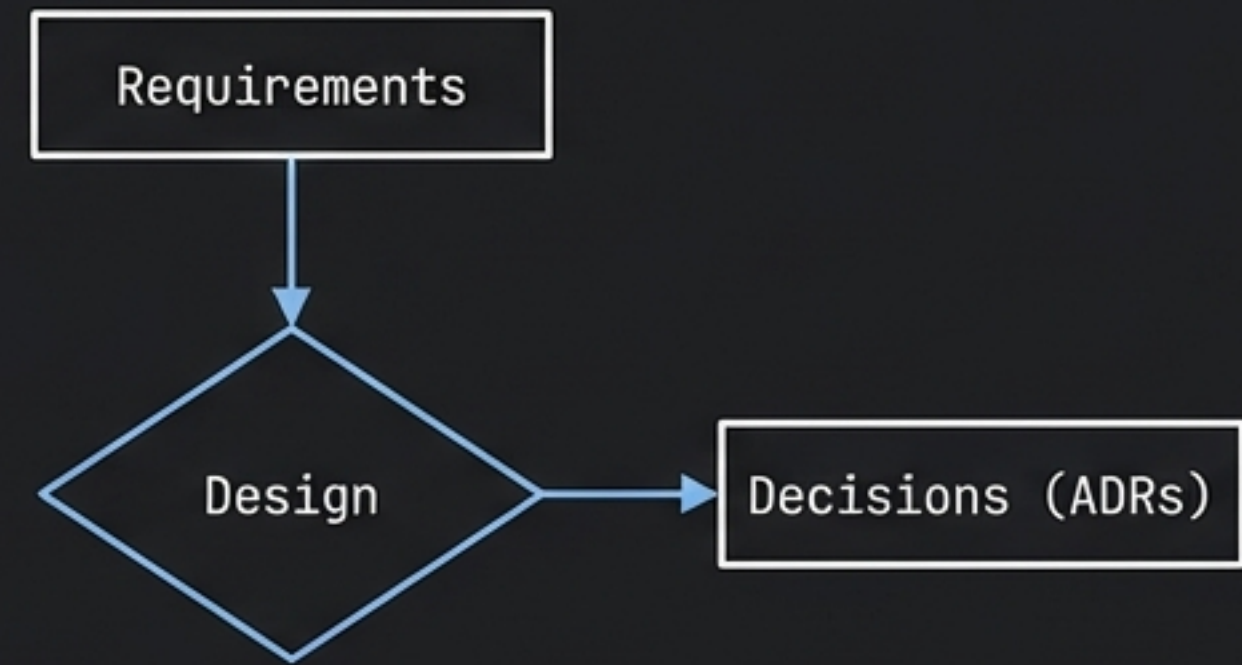
Operational Management



- Owns the Sprint
- Hands off to: All Roles
- Distinction: Not a Project Manager. Orchestrates without doing the work.

The Architect

Technical Strategy



- Owns the Decision
- Hands off to: Dev, Librarian
- Key Artifact: Architecture Decision Records

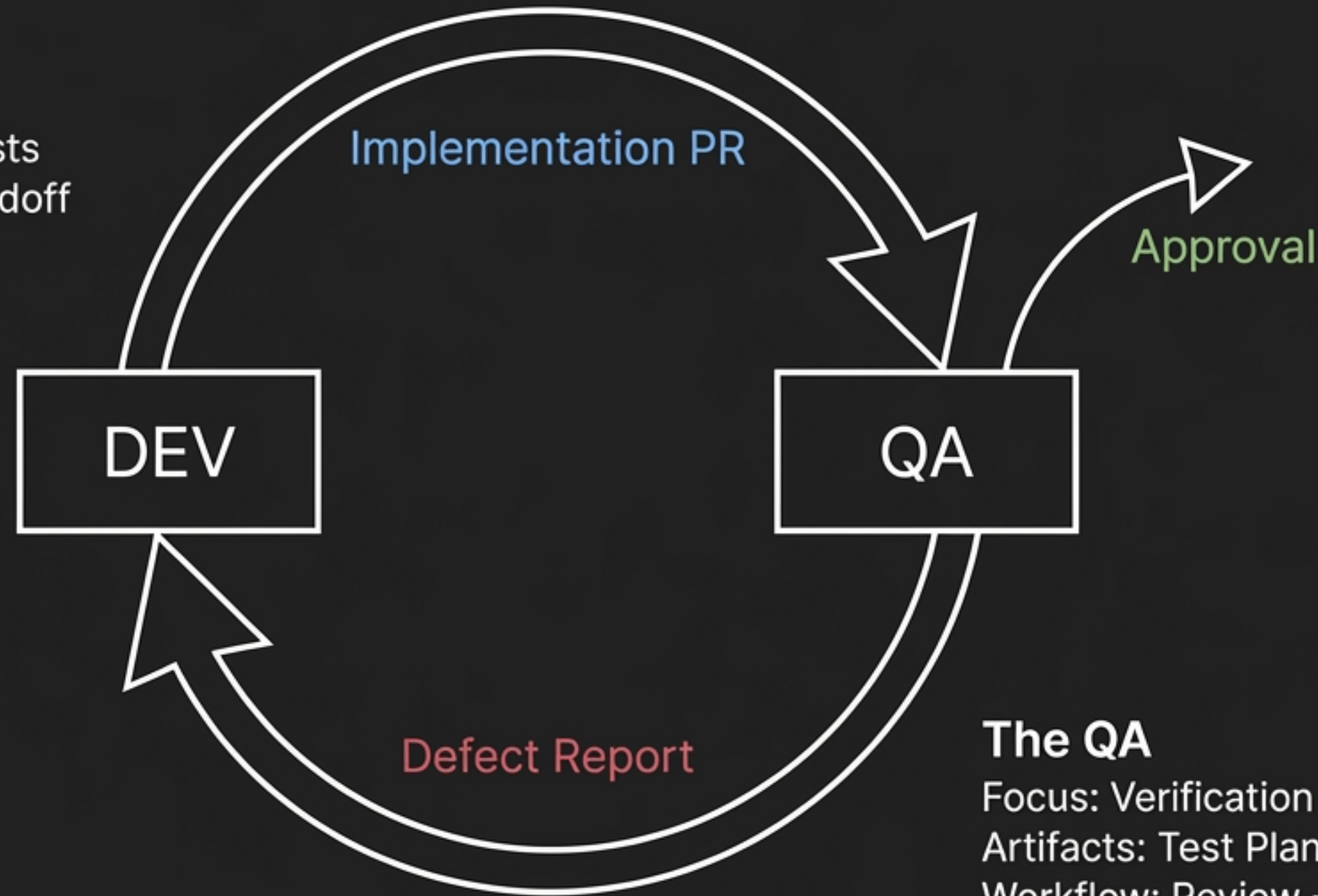
Execution Roles: Dev & QA

The Dev

Focus: Implementation

Artifacts: Source Code, Unit Tests

Workflow: Task -> Code -> Handoff



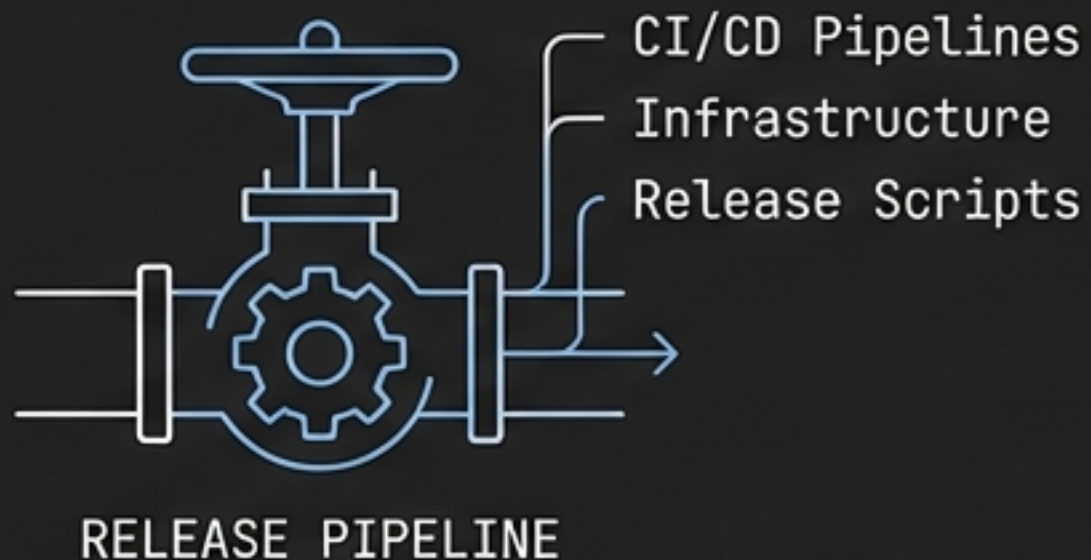
The QA

Focus: Verification

Artifacts: Test Plans, Integration Tests

Workflow: Review -> Approve OR Defect

Support Roles: DevOps & Librarian



DevOps: The Gatekeeper

Responsibilities: CI/CD Pipelines, Infrastructure, Release Scripts.

Authority: The only role that can trigger a **Release issue**. Protects production from **bad code**.



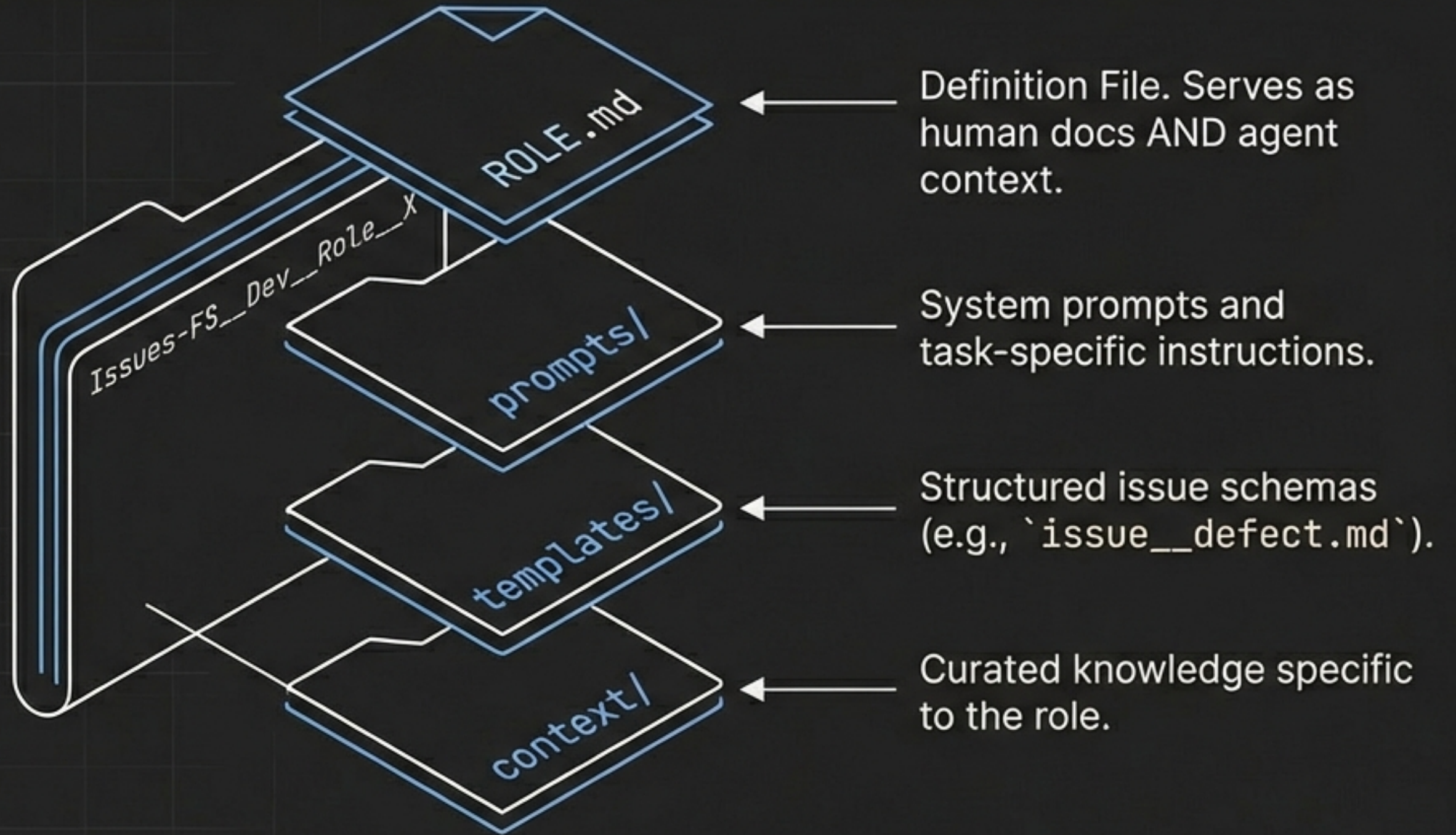
Librarian: The Curator

Responsibilities: Knowledge Coherence, Changelogs, Onboarding.

Philosophy: "Docs" implies a static dump. "Librarian" implies judgment—deciding what is **canonical** and what is **stale**.

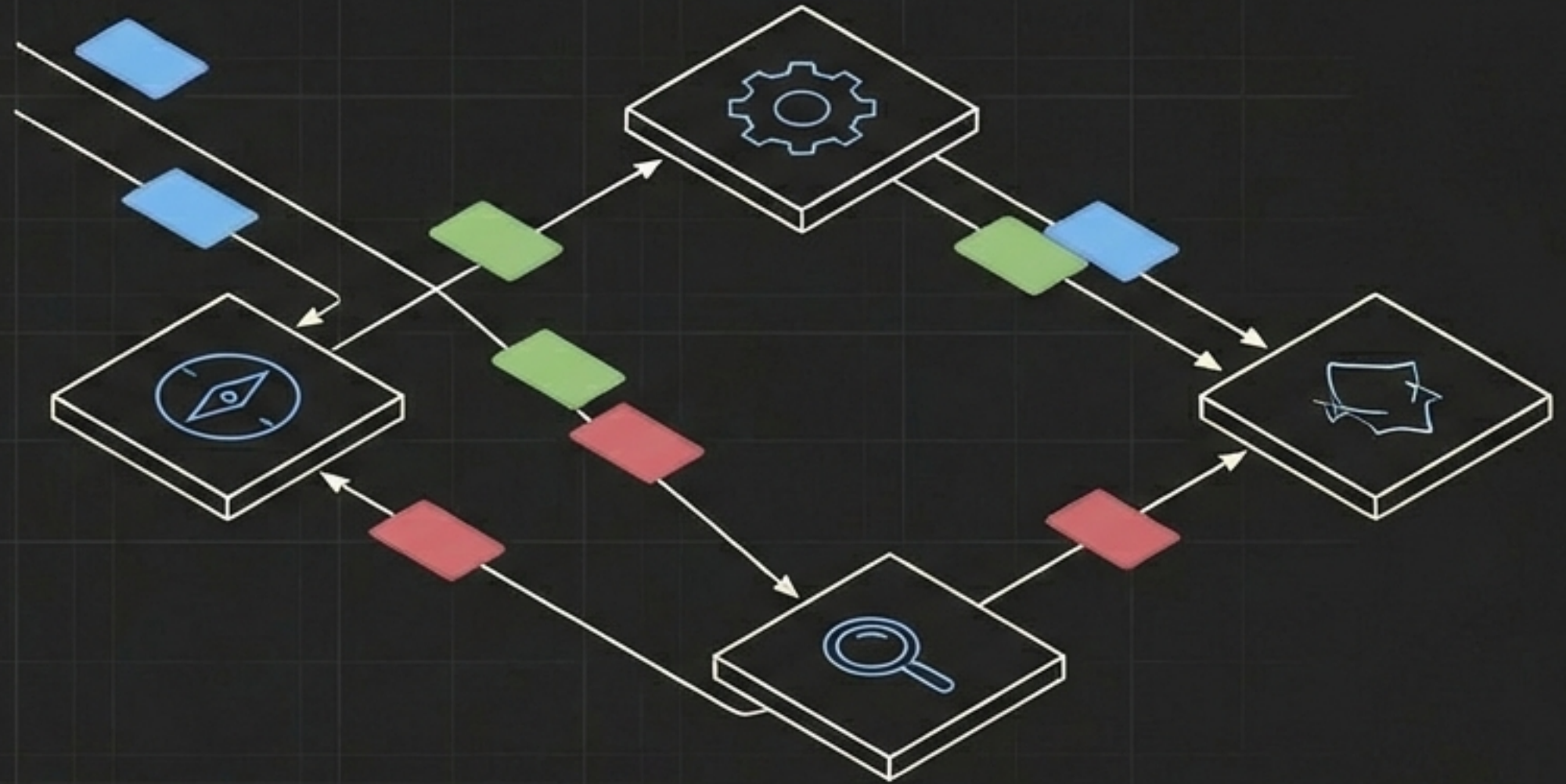
Anatomy of a Role Repo

A role repo encodes responsibilities, boundaries, and coordination protocols.



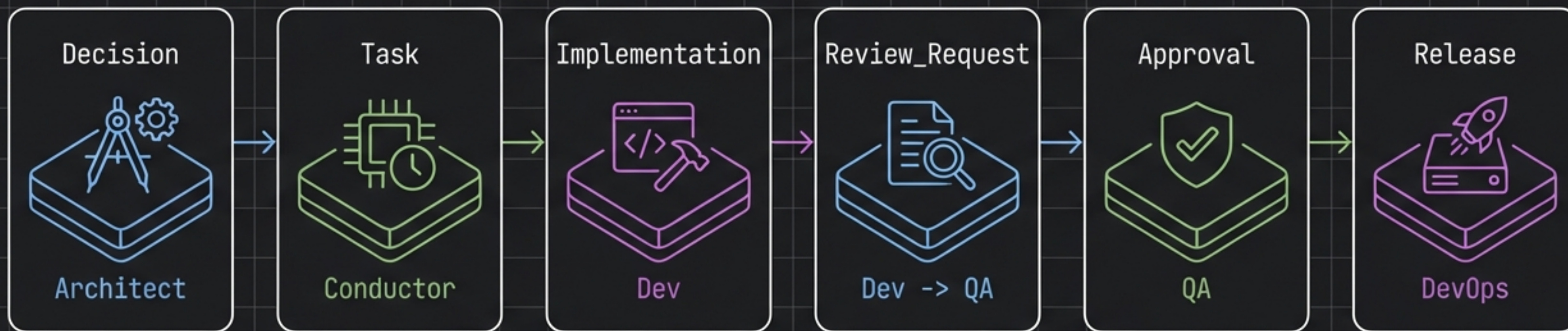
Coordination via Typed Issues

1. **Decision** (Architect)
-> Strategic Choice
2. **Handoff** (Any)
-> Transfer of Work
3. **Review_Request** (Any)
-> Asking for Approval
4. **Defect** (QA)
-> Bug Report
5. **Blocker** (Any)
-> Escalation
6. **Release** (DevOps)
-> Deployment Candidate



"Issues are the '**State Machine**'. Communication happens via **structured data**, not chat logs."

The Workflow State Machine



Traceability: The entire flow is tracked as a graph query.
"Show me all issues linked to this Decision".

Dogfooding as Architecture

Concept:

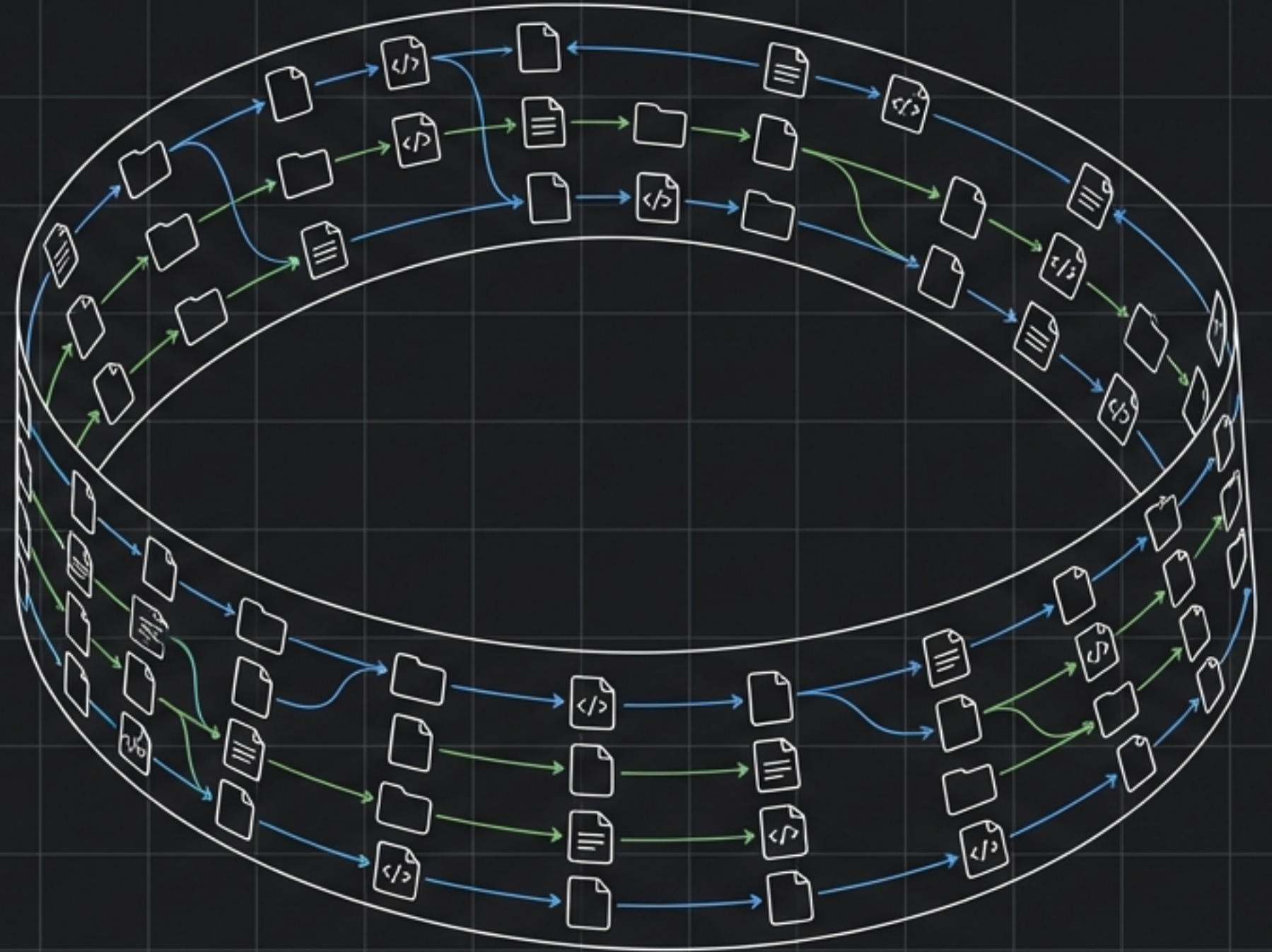
Issues-FS manages itself using **Issues-FS**.

Mechanism:

- Each **role repo** tracks its own work as **Issues-FS issues**.
- Cross-role coordination uses the **Issues-FS graph**.

Impact:

Improvements to the **platform** immediately improve the process of building the **platform**.

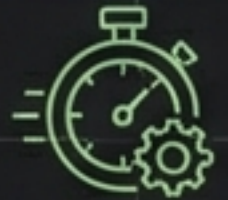


Scaling the Ecosystem

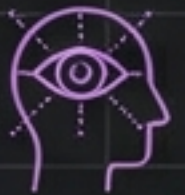
Future Roles



Security: Threat modeling, auditing



Performance: Benchmarking, SLAs



UX: User research, Design

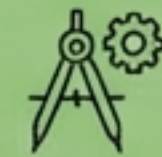


Data: Schema evolution

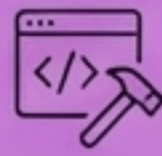
Role Addition Protocol



Decision (Conductor)



Evaluation (Architect)



Creation (Dev)



Documentation (Librarian)

Guiding Principle: Create highly focused teams with just the resources needed.

Key Architectural Decisions (ADRs)

D1: Role vs. Persona	Roles define ownership. Personas define tone. In coordination, ownership matters more.
D2: Conductor vs. PM	“Project Manager” carries baggage. “Conductor” implies orchestration of an ensemble without doing the grunt work.
D4: Decisions are Issues	Every decision gets a unique ID and history. This enables graph queries on the decision tree itself.
D5: Typed Issues	The issue schema IS the workflow engine. No external orchestration tool is needed.

Summary & Status

Document: issues-fs__role-based-agent-coordination
Version: v1.0 (Draft)
Date: 2026-02-05

Dependencies:

- Issues-FS__Architecture-Overview
- Memory-FS
- MGraph-DB
- OSBot-Utils

The submodule pattern works for code. The insight is that it works equally well for agent roles.